RETAIL DATA AGENT · REQUIREMENT 1 · AS OF 2026-11-01

DESIGN ONLY · NOT BUILT IN THE PROTOTYPE

How does the agent learn from analysts' past work?

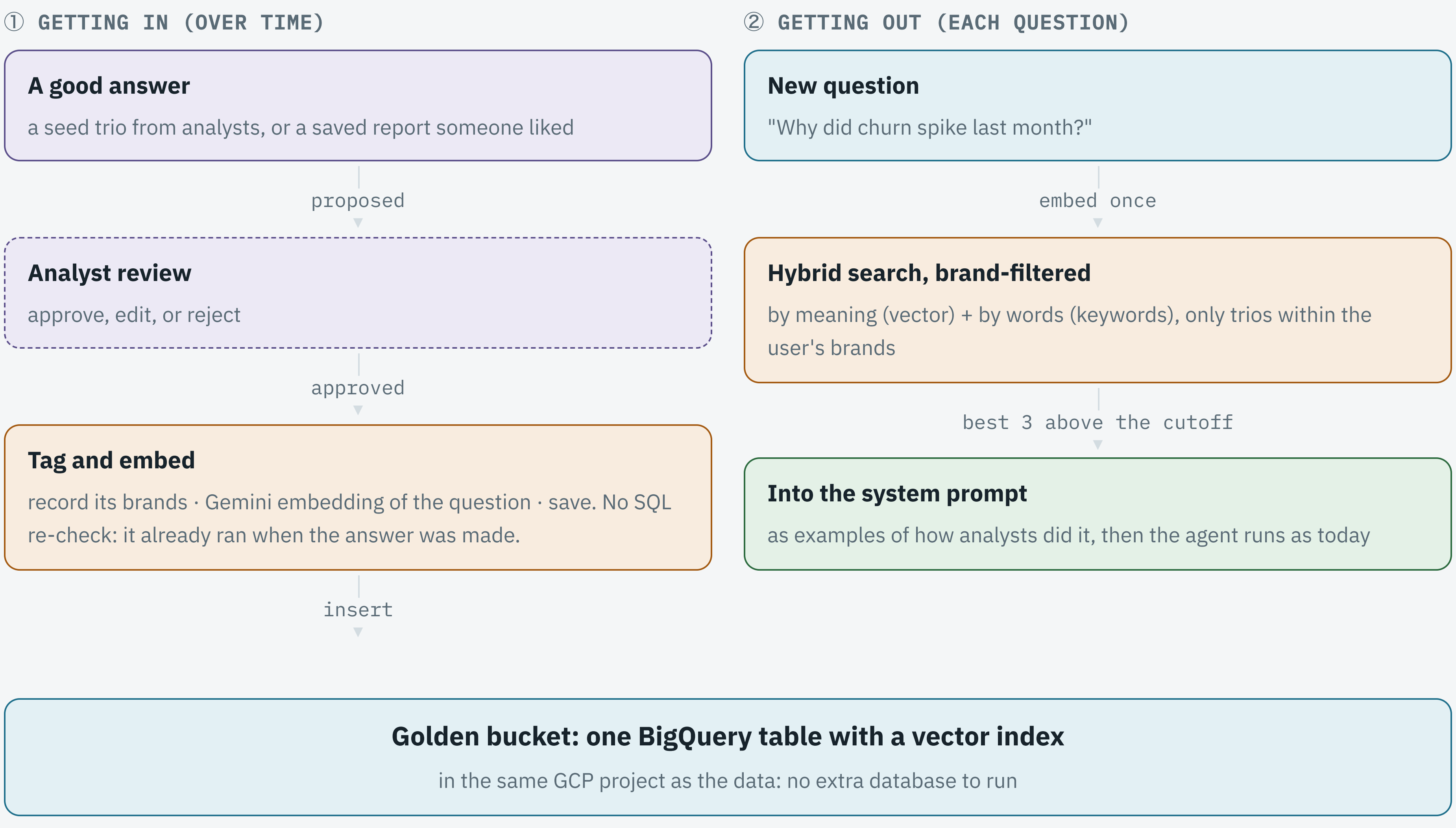
Analysts have answered many questions before. Each answer is saved as a **trio**: the question, the SQL they wrote, and their written answer. Together the trios are the **Golden Knowledge** bucket. Before the agent writes any SQL, it looks up the few trios most like the new question and follows the analysts' approach, but only trios the user is allowed to see. See also the architecture and brand scoping.

🟪 an analyst decides 🟦 stored in BigQuery 🟧 embedding and search 🟩 reaches the prompt 🟥 filtered out

In short

1. **Getting in:** an analyst approves a good answer, and it's stored as a trio with its brands and a vector (a list of numbers that captures the question's meaning).
2. **Getting out:** before the first model call, the question is searched two ways at once, by meaning and by exact words, and the best 3 trios go into the prompt.
3. **Same access rules:** the search only looks at trios within the user's brands, the same rule as the query gateway.
4. **Kept fresh:** every night each trio's SQL is dry-run; one that breaks is set aside for an analyst.

The picture: two lanes into one bucket

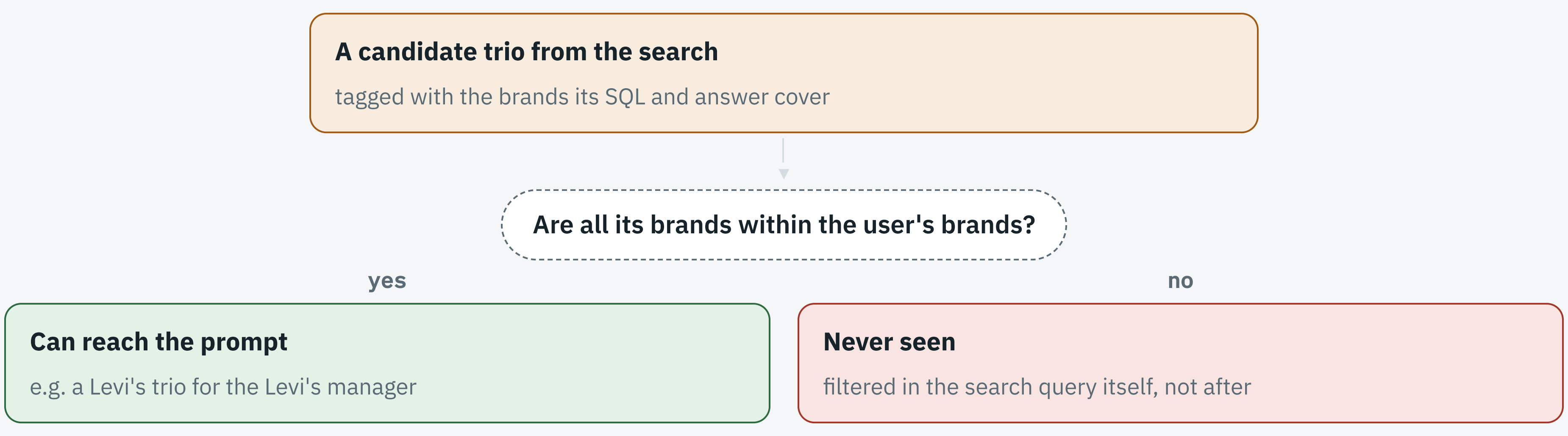


① GETTING IN (OVER TIME)
② GETTING OUT (EACH QUESTION)
A good answer
a seed trio from analysts, or a saved report someone liked
proposed
Analyst review
approve, edit, or reject
approved
Tag and embed
record its brands · Gemini embedding of the question · save. No SQL re-check: it already ran when the answer was made.
insert
New question
"Why did churn spike last month?"
embed once
Hybrid search, brand-filtered
by meaning (vector) + by words (keywords), only trios within the user's brands
best 3 above the cutoff
Into the system prompt
as examples of how analysts did it, then the agent runs as today
Golden bucket: one BigQuery table with a vector index
in the same GCP project as the data: no extra database to run

🏠 Like a new analyst's folder of "how we solved it before". The team lead adds a solution to the folder (lane ①); before starting a task, the analyst pulls out the 3 most similar ones (lane ②).

The guardrail: a trio holds numbers, so it follows brand scope

A trio's written answer contains real figures. If the Levi's manager's question matched a CEO trio about Calvin Klein, those figures would land in the prompt and could leak. So the search filters trios exactly as the gateway filters rows.



A candidate trio from the search
tagged with the brands its SQL and answer cover
Are all its brands within the user's brands?
yes
no
Can reach the prompt
e.g. a Levi's trio for the Levi's manager
Never seen
filtered in the search query itself, not after

An analyst can also mark a trio **method only**: the figures are removed from its answer, so only the approach is left, such as how churn is defined. Then it's safe for every user.

Example: the Levi's manager asks "Why did churn spike last month?"

1. The question is embedded once (one embedding call, which is not a chat call and doesn't count toward the 5-call budget).
2. One BigQuery query searches by meaning and by words, keeps the trios this user may see, merges the two rankings and keeps those above the cutoff:

TRIO	BRANDS	MATCH	RESULT
"Churn rate by month for Levi's buyers"	Levi's	0.91	✅ added
"How we define churn" (method only)	method only	0.88	✅ added
"Calvin Klein churn after the price change"	Calvin Klein	0.90	❌ not this user's brand
"Monthly revenue by category"	all	0.41	below the cutoff

3. The two kept trios go into the system prompt under a heading like "How analysts answered similar questions":

```
# added to the system prompt, before the first model call
How analysts answered similar questions (examples, not current data):
Q: How we define churn
SQL: customers with an order in the 3 months before, and none last month …
A: Churn = share of those customers who didn't buy last month. Compare with the same month last year.
```

4. The agent writes its SQL the analysts' way. That SQL still goes through the query gateway like any other: the trios guide the model, they don't bypass any check.

Numbers made up to show the idea.

The limits it keeps

0
extra LLM calls
Search runs before the first call, so a question still costs 2–5 calls. A "search" tool would add one each time.

3
trios at most
Enough to show the method, few enough to keep the prompt short and cheap.

cutoff
or nothing
If no trio is similar enough, none is added. A weak example misleads more than no example.

nightly
freshness check
Each trio's SQL is dry-run (free). One that fails after a schema change goes back to review.

If the search fails or is slow, the agent simply answers without examples, as it does today. Golden Knowledge makes answers better; it is never a single point of failure.

What one trio stores

FIELD	WHAT IT HOLDS	WHY
`question`	The question, as asked	What new questions are compared against
`sql`	The SQL the analyst used	The method to copy, and what the nightly dry run checks
`answer`	The analyst's written answer: one line, a paragraph or a full report	How to read the numbers, not just how to fetch them
`brands`	The brands it covers, or "method only"	The brand filter in the search
`embedding`	Gemini embedding of the question	Search by meaning, through a BigQuery vector index
`status`	active, or flagged for review	Only active trios are searched
`approved_by, approved_at`	Who approved it, and when	Accountability, and to retire old ones

> **The rule in one sentence**
>
> Only analyst-approved trios within the user's brands, at most 3 and only when similar enough, are added to the prompt before the first call. They guide how the agent works; every query it writes still goes through the gateway.

Status: design only (brief requirement 1; the PDF calls the bucket "theoretical"). Section "Golden Knowledge" in `docs/hld.md`.

Services: Gemini embedding model (Vertex AI, or `ML.GENERATE_EMBEDDING` in BigQuery); BigQuery table `golden.trios` with `CREATE VECTOR INDEX` on `embedding` and a search index on `question`; retrieval in one query, with `VECTOR_SEARCH` for meaning, `SEARCH` for keywords, rankings merged by reciprocal rank fusion. Nightly freshness check as a BigQuery scheduled query or Cloud Scheduler job. Vertex AI Vector Search is the step up if the bucket grows past what BigQuery serves fast enough.

Where it plugs into today's code: retrieval runs in `App.ask` (or a `before_model` middleware on the first call) and fills a `{golden-examples}` placeholder in `prompts/system.md`, filled the same way as `{user-brands}`. The brand filter reuses `User.brands` from `users.yaml`. Candidates come from the existing report store (`reports.py`), which already keeps question, SQL and answer together.

See also architecture, brand scoping, report queries.